

분류

- K-최근접 이웃(K-Nearest
Neighbors)

1. 마켓과 머신러닝

머신러닝 모델 훈련해보기



- **k-최근접 이웃을 사용하여 2개의 종류를 분류하는 머신러닝 모델을 훈련해보기**

- 학습의 가상 조건 설정

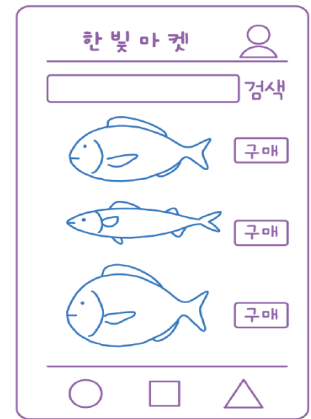
- 한빛 마켓은 앱 마켓 최초로 살아 있는 생선을 판매하기 시작
- 고객이 온라인으로 주문하면 가장 빠른 물류 센터에서 신선한 생선을 곧바로 배송

- 문제 발생

- 물류 센터에서 생선을 고르는 직원이 도통 생선 이름을 외우지 못함
- 항상 주위 사람에게 "이 생선 이름이 뭐예요?"라고 물어봐 배송이 지연되기 일쑤

- 문제 해결

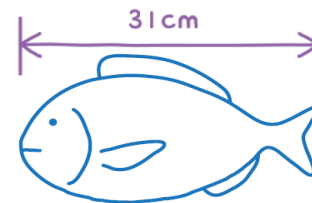
- **혼공머신에게 첫 번째 임무로 생선 이름을 자동으로 알려주는 머신러닝을 만들기**



생선 분류 문제



- 한빛 마켓에서 팔기 시작한 생선은 ‘도미’, ‘곤들매기’, ‘농어’, ‘강꼬치 고기’, ‘로치’, ‘빙어’, ‘송어’
- 이 생선들은 물류 센터에 많이 준비되어 있고, 이 생선들을 프로그램으로 분류한다고 가정하여 적합한 프로그램을 만들기
- 사용할 생선 데이터는 캐글에 공개된 데이터셋
 - <https://www.kaggle.com/aungpyaeap/fish-market>
 - 캐글(kaggle.com)은 2010년에 설립된 전 세계에서 가장 큰 머신러닝 경연 대회 사이트로, 대회 정보뿐만 아니라 많은 데이터와 참고 자료를 제공



생선 분류 문제



- 생선을 분류하는 일이니 생선의 특징을 알면 쉽게 구분할 수 있을 것으로 예상
- 도미 특성1: 생선 길이가 30cm 이상이면 도미

```
if fish_length >= 30:  
    print("도미")
```

- 문제 발생
 - 30cm보다 큰 생선이 무조건 도미? 또 도미의 크기가 모두 같을 리도 없음
- 한빛 마켓에서 판매하는 생선은 이렇게 절대 바뀌지 않을 기준을 정하기 어려움
- 이 문제를 머신러닝으로 어떻게 해결할 수 있을까?
 - 보통 프로그램은 '누군가 정해진 기준대로 일'을 수행
 - 반대로 머신러닝은 누구도 알려주지 않는 기준을 찾아서 일을 수행
즉, 누가 말해 주지 않아도 머신러닝은 "30~40cm 길이의 생선은 도미이다"라는 기준을 찾음
- 머신러닝은 기준을 찾을 뿐만 아니라 이 기준을 이용해 생선이 도미인지 아닌지 판별

생선 분류 문제



- 도미 데이터 준비하기

- 머신러닝은 어떻게 이런 기준을 스스로 찾을 수 있을까?
 - 머신러닝은 여러 개의 도미 생선을 보면 스스로 어떤 생선이 도미인지를 구분할 기준을 찾음
 - 그렇다면 도미 생선을 많이 준비해야 함

- 무게와 길이를 함께 재어 주는 저울을 이용하여 데이터 수집

- 먼저 머신러닝을 사용해 도미와 빙어를 구분
- 도미와 빙어를 준비해서 저울에 올려놓고 무게와 길이를 측정

생선 분류 문제



- 도미 데이터 준비하기
 - 35마리의 도미를 준비하고 저울로 잰 도미의 길이(cm)와 무게(g)를 파이썬 리스트로 준비

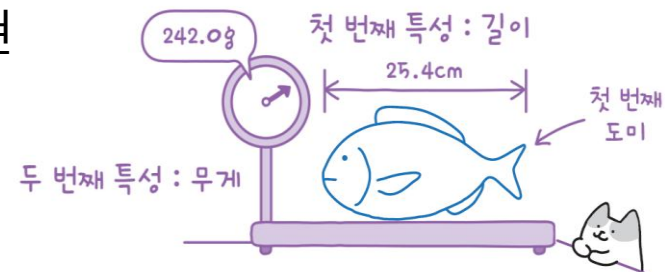
```
bream_length = [ 25.4, 26.3, 26.5, 29.0, 29.0, 29.7, 29.7, 30.0, 30.0, 30.7,  
                 31.0, 31.0, 31.5, 32.0, 32.0, 32.0, 33.0, 33.0, 33.5, 33.5,  
                 34.0, 34.0, 34.5, 35.0, 35.0, 35.0, 35.0, 36.0, 36.0, 37.0,  
                 38.5, 38.5, 39.5, 41.0, 41.0]
```

← 생선의 길이

```
bream_weight = [ 242.0, 290.0, 340.0, 363.0, 430.0, 450.0, 500.0, 390.0,  
                 450.0, 500.0, 475.0, 500.0, 500.0, 340.0, 600.0, 600.0,  
                 700.0, 700.0, 610.0, 650.0, 575.0, 685.0, 620.0, 680.0,  
                 700.0, 725.0, 720.0, 714.0, 850.0, 1000.0, 920.0, 955.0,  
                 925.0, 975.0, 950.0]
```

← 생선의 무게

- 리스트에서 첫 번째 도미의 길이는 25.4cm, 무게는 242.0g이고 두 번째 도미의 길이는 26.3cm, 무게는 290.0g
- 특성(feature): 각 도미를 길이와 무게로 표현



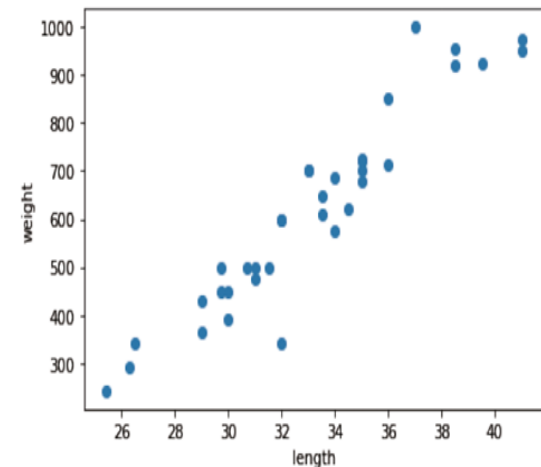
생선 분류 문제



- 산점도(scatter plot)
 - 두 특성을 숫자에서 그래프로 표현
 - 길이를 x축, 무게를 y축
 - 각 도미를 이 그래프에 점으로 표시
 - 맷플로립의 scatter() 함수 사용

```
import matplotlib.pyplot as plt # matplotlib의 pyplot 함수를 plt로 줄여서 사용

plt.scatter(bream_length, bream_weight)
plt.xlabel('length') # x축은 길이
plt.ylabel('weight') # y축은 무게
plt.show()
```



생선 분류 문제

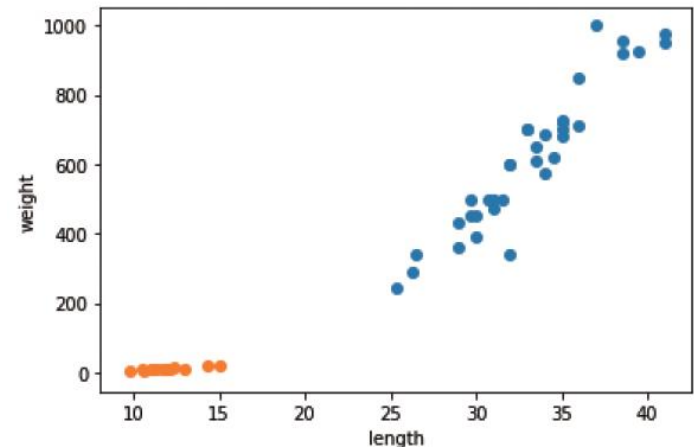


- 빙어 데이터 준비하기
 - 빙어 14마리의 데이터를 앞에서와 같이 파이썬 리스트로 만들기

```
smelt_length = [ 9.8, 10.5, 10.6, 11.0, 11.2, 11.3, 11.8, 11.8, 12.0, 12.2,  
                12.4, 13.0, 14.3, 15.0]  
smelt_weight = [ 6.7, 7.5, 7.0, 9.7, 9.8, 8.7, 10.0, 9.9, 9.8, 12.2, 13.4,  
                12.2, 19.7, 19.9]
```

- 빙어의 산점도 그래프
 - scatter() 함수를 연달아 사용하여 2개의 산점도를 한 그래프로 그리기

```
plt.scatter(bream_length, bream_weight)  
plt.scatter(smelt_length, smelt_weight)  
plt.xlabel('length')  
plt.ylabel('weight')  
plt.show()
```



첫 번째 머신러닝 프로그램



- k-최근접 이웃(k-Nearest Neighbors) 알고리즘을 사용해 도미와 빙어 데이터를 구분
 - k-최근접 이웃 알고리즘을 사용하기 전에 앞에서 준비했던 도미와 빙어 데이터를 하나의 데이터로 합침
 - 파이썬에서는 다음처럼 두 리스트를 더하면 하나의 리스트로 만들어 줌

```
length = bream_length + smelt_length  
weight = bream_weight + smelt_weight
```

도미 35개의 길이 빙어 14개의 길이

length = [25.4, 26.3, ... , 41.0, 9.8, ... , 15.0]

도미 35개의 무게 빙어 14개의 무게

weight = [242.0, 290.0, ... , 950.0, 6.7, ... , 19.9]

첫 번째 머신러닝 프로그램



- **fish_data 만들기 : [길이, 무게]**

- 사이킷런(scikit-learn) 패키지를 사용하려면
오른쪽처럼 각 특성의 리스트를 세로 방향으로
늘어뜨린 **2차원 리스트**를 만들어야 함

49개의 생선

길이	무게
25.4	242.0
26.3	290.0
.	.
.	.
.	.
15.0	19.9

- 파이썬의 zip() 함수와 리스트 내포 구문을 사용
 - zip() 함수는 나열된 리스트 각각에서 하나씩 원소를 꺼내 반환
 - zip() 함수와 리스트 내포 구문을 사용해 length와 weight 리스트를 2차원 리스트로 만들기

```
fish_data = [[l, w] for l, w in zip(length, weight)]
```

- for 문은 zip() 함수로 length와 weight 리스트에서 원소를 하나씩 꺼내어 l
과 w에 할당하면 [l, w]가 하나의 원소로 구성된 리스트가 만들어짐

- [1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

첫 번째 머신러닝 프로그램



- 사이킷런 패키지에서 k-최근접 이웃 알고리즘을 구현한 클래스인 **KNeighborsClassifier**를 **임포트**

```
from sklearn.neighbors import KNeighborsClassifier
```

- 임포트한 **KNeighborsClassifier** 클래스의 객체 만들기

```
kn = KNeighborsClassifier()
```

- 훈련(training) – fit() 메소드**

- 이 객체에 fish_data와 fish_target을 전달하여 학습시킴
- 이 메서드에 fish_data와 fish_target을 순서대로 전달

```
kn.fit(fish_data, fish_target)
```

- 객체(또는 모델) kn이 얼마나 잘 훈련되었는지 평가**

```
kn.score(fish_data, fish_target)
```



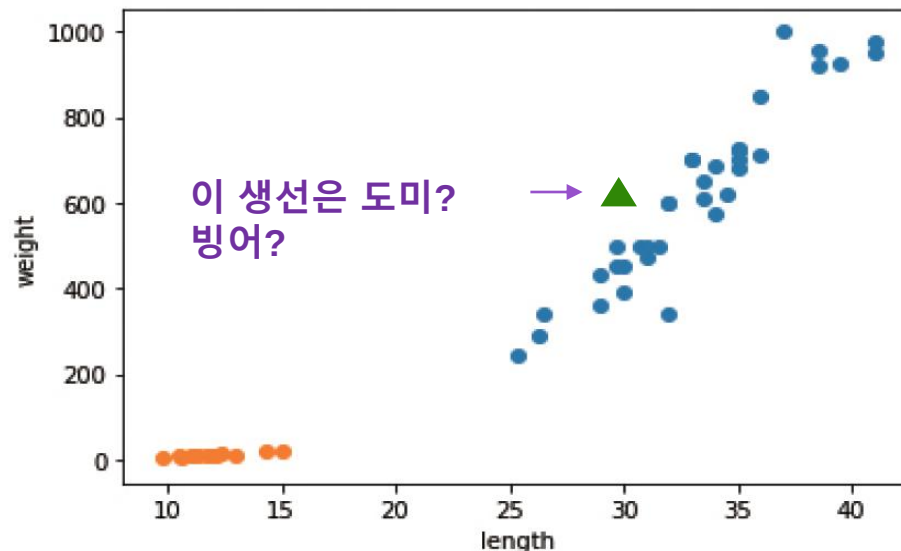
1.0

첫 번째 머신러닝 프로그램



▪ k-최근접 이웃 알고리즘

- 어떤 데이터에 대한 답을 구할 때 주위의 다른 데이터를 보고 다수를 차지하는 것을 정답으로 사용
- 다음 그림에 삼각형으로 표시된 새로운 데이터가 있다고 가정하면, 이 삼각형은 도미와 빙어 중 어디에 속할까?



첫 번째 머신러닝 프로그램



▪ k-최근접 이웃 알고리즘

- predict() 메서드는 새로운 데이터의 정답을 예측
- fit() 메서드와 마찬가지로 2차원 리스트 전달해야 함
그래서 삼각형 포인트를 리스트로 2번 감싸고, 반환되는 값은 1
- 도미는 1, 빙어는 0으로 가정. 따라서 삼각형(▲)은 도미

```
kn.predict([[30, 600]])
```



```
array([1])
```

첫 번째 머신러닝 프로그램



▪ k-최근접 이웃 알고리즘

- 새로운 데이터에 대해 예측할 때는 가장 가까운 직선거리에 어떤 데이터가 있는지를 살피기만 하면 됨
- 단점은 k-최근접 이웃 알고리즘의 이런 특징 때문에 데이터가 아주 많은 경우 사용하기 어려움
데이터가 크기 때문에 메모리가 많이 필요하고, 직선거리를 계산하는데도 많은 시간이 필요

첫 번째 머신러닝 프로그램



- **k-최근접 이웃 알고리즘**

- **fit() 메서드**에 전달한 데이터를 모두 저장하고 있다가 새로운 데이터가 등장하면 가장 가까운 데이터를 참고하여 도미인지 빙어인지를 구분
- 가까운 몇 개의 데이터를 참고할지는 정하기 나름이지만, **KNeighborsClassifier 클래스의 기본값은 5**임
이 기준은 n_neighbors 매개변수로 바꿀 수 있음

첫 번째 머신러닝 프로그램



▪ k-최근접 이웃 알고리즘

- 다음과 같이 하면 어떤 결과가 나올까?

```
kn49 = KNeighborsClassifier(n_neighbors=49)
```

참고 데이터를 49개로 한 kn49 모델

- 가장 가까운 데이터 49개를 사용하는 k-최근접 이웃 모델에 fish_data를 적용하면 fish_data에 있는 모든 생선을 사용하여 예측
 - fish_data의 데이터 49개 중에 도미가 35개로 다수를 차지하므로 어떤 데이터를 넣어도 무조건 도미로 예측

```
kn49.fit(fish_data, fish_target)  
kn49.score(fish_data, fish_target)
```



0.7142857142857143

도미와 빙어 분류(문제해결 과정)



▪ 분류 준비 과정

- 도미와 빙어를 구분하기 위한 첫 번째 머신러닝 프로그램을 작성
- 먼저 도미 35마리와 빙어 14마리의 길이와 무게를 측정해서 파이썬 리스트 생성
- 도미와 빙어 데이터를 합쳐 2차원 리스트 데이터를 준비

▪ 첫 번째 머신러닝 프로그램

- k-최근접 이웃 알고리즘
- 사이킷런의 k-최근접 이웃 알고리즘은 주변에서 가장 가까운 5개의 데이터를 보고 다수결의 원칙에 따라 데이터를 예측

▪ 사용 메서드

- KNeighborsClassifier 클래스의 fit (), score (), predict () 메서드 사용

키워드로 끝내는 핵심 포인트



- 특성은 데이터를 표현하는 하나의 성질. 이 절에서 생선 데이터 각각을 길이와 무게 특성으로 표현
- 머신러닝 알고리즘이 데이터에서 규칙을 찾는 과정을 훈련
 - 사이킷런에서는 `fit()` 메서드가 하는 역할
- k-최근접 이웃은 가장 간단한 머신러닝 알고리즘 중 하나
- 머신러닝 프로그램에서 알고리즘이 구현된 객체를 모델이라 함
- 정확도는 정확한 답을 몇 개 맞혔는지를 백분율로 나타낸 값
 - 사이킷런에서는 0 ~1 사이의 값으로 출력
 - $\text{정확도} = (\text{정확히 맞힌 개수}) / (\text{전체 데이터 개수})$

확인 문제



1. 데이터를 표현하는 하나의 성질로써, 예를 들어 국가 데이터의 경우 인구 수, GDP, 면적 등이 하나의 국가를 나타냄. 머신러닝에서 이런 성질을 무엇이라고 하나?

- ① 특성 ② 특질 ③ 개성 ④ 요소

2. 가장 가까운 이웃을 참고하여 정답을 예측하는 알고리즘이 구현된 사이킷런 클래스는?

- ① SGDClassifier ② LinearRegression
③ RandomForestClassifier ④ KNeighborsClassifier

확인 문제



3. 사이킷런 모델을 훈련할 때 사용하는 메서드는 어떤 것인가?

- ① predict() ② fit()
- ③ score() ④ transform()

4. 다음 중 모델의 정확도를 계산하는 올바른 방법은?

- ① (틀린 개수) / (전체 데이터 개수)
- ② (전체 데이터 개수) / (틀린 개수)
- ③ (정확히 맞힌 개수) / (전체 데이터 개수)
- ④ (전체 데이터 개수) / (정확히 맞힌 개수)



Q & A

